

0.79. Suivre une autre lettre

Le A suit la croix ; le B le poursuit tout seul : à chaque pas, il avance vers le A, de ± 1 sur chaque axe.

1. Ce qu'il faut comprendre

Un poursuivant : tu déplaces le A avec la croix ; le B avance tout seul **vers le A**, un pas tous les **600 ms**. C'est le premier « ennemi ».

Ce qui est nouveau ici : choisir la direction en comparant. Pour la colonne : si le A est plus à droite ($x > bx$), un pas à droite ($sx = 1$) ; plus à gauche, un pas à gauche ($sx = -1$) ; même colonne, rien ($sx = 0$). Pareil pour la ligne avec sy .

`un_pas(bx, by, ALPHABET[1], sx, sy)` (le 0.30) fait ce pas. Avec sx et sy à la fois, le B va **en diagonale** vers le A.

Attrapé : quand le B arrive sur la case du A, le mot `PRIS` s'écrit en bas.

Pourquoi 600 ms : le A fait un pas tous les 250 ms, plus de **deux fois plus vite** que le B. On peut donc lui échapper. Mais le B avance aussi en diagonale : dans un coin, il finit par nous coincer. Plus rapide (300 ms), il serait presque impossible à fuir. Le 0.79.1 le fait accélérer peu à peu.

L'ordre compte : le B bouge **avant** que la croix ne repose le A. Ainsi le A est dessiné par-dessus, et reste visible même quand le B le touche.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <un_pas>` → [0.29.4. La fonction un_pas\(\) — un seul pas, sans attendre](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1590 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          85 o ← la console, à l'allumage — la mise en route de la console
chaque_minuteur()  124 o ← main() — une fonction fournie par le langage
deplace_croix()    217 o ← main() — une fonction fournie par le langage
un_pas()           117 o ← main() — une fonction fournie par le langage
main()             174 o ← démarrage — ta fonction
VBlank             8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o ← image() — attend l'image suivante
AttendreVBlank     16 o ← démarrage — attend que l'écran soit libre
AttendreAcces      12 o ← EffacerCases, EcrireTexte — attend de pouvoir écrire une case
```

AdresseCase	10 o	← effacer(), poser() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	704 o	← CopierTuiles – la police entière
"PRIS"	4 o	← texte() – un texte gravé

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

effacer()	x 2	30 o sur place, dans deplace_croix(), un_pas() + AdresseCase (10 o), EffacerCases (12 o)
poser()	x 2	26 o sur place, dans deplace_croix(), un_pas() + AdresseCase (10 o)
bouton()	x 4	20 o sur place, dans deplace_croix() + LireManette (37 o)
texte()	x 1	11 o sur place, dans main() + EcrireTexte (10 o)
images()	x 2	4 o sur place, dans chaque_minuteur(), deplace_croix()
image()	x 1	3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

un_pas()	x 1	23 o sur place, dans main()
deplace_croix()	x 1	19 o sur place, dans main()
chaque_minuteur()	x 1	10 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <chaque>           // répond 1 toutes les n millisecondes
#include <un_pas>            // fait faire un seul pas à une tuile
#include <ALPHABET>         // les lettres de la police : ALPHABET[0] est le A
#include <deplace_croix>    // une tuile qui suit la croix, case par case
#include <texte>             // écrit un texte à l'écran
```

2. UNE VARIABLE — X

```
uint8_t x = 10;           // le A (la croix)
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;
```

4. UNE VARIABLE — BX

```
uint8_t bx = 0;           // le B (le poursuivant) : il part du coin (0, 0)
```

5. UNE VARIABLE — BY

```
uint8_t by = 0;
```

6. UNE VARIABLE — SX

```
uint8_t sx = 0;           // la direction du prochain pas du B, en colonne...
```

7. UNE VARIABLE — SY

```
uint8_t sy = 0;           // ...et en ligne : 1, -1 ou 0
```

8. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();

        if (chaque(600)) {                // un pas du B tous les 600 ms : plus lent que le A
            // La direction : vers le A, sur chaque axe.
            sx = 0;
            if (x > bx) { sx = 1; }        // le A est à droite : un pas à droite
            if (x < bx) { sx = -1; }       // à gauche : un pas à gauche
            sy = 0;
            if (y > by) { sy = 1; }        // plus bas : un pas en bas
            if (y < by) { sy = -1; }       // plus haut : un pas en haut
            if (sx != 0 || sy != 0) {      // pas déjà sur lui (|| : OU)
                un_pas(bx, by, ALPHABET[1], sx, sy);
            }
        }

        deplace_croix(x, y, ALPHABET[0], 250); // le A, dessiné après : par-dessus

        if (bx == x && by == y) {         // le B est sur la case du A
            texte(0, 17, "PRIS");
        }
    }
}
```

3. Ce qui se passe, étape par étape

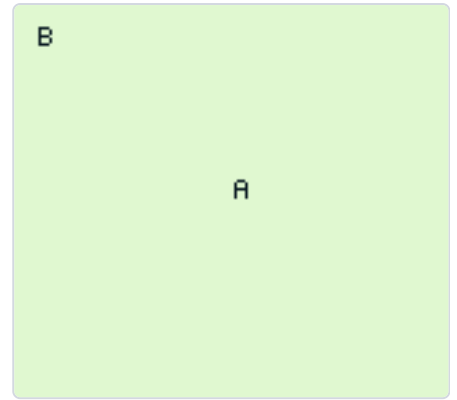
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



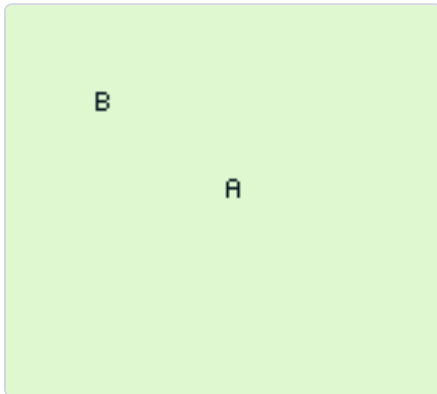
la toute première image — la console vient de démarrer — image 1



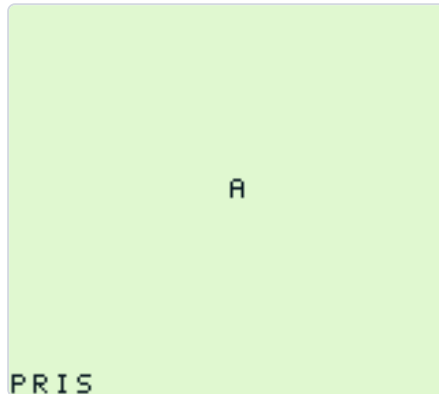
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A au milieu, un B dans le coin qui s'approche tout seul ; déplace le A avec les flèches pour lui échapper. S'il t'attrape, PRIS.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le B se rapproche du A (distance 18 → 12)

✓ immobile, le A finit attrapé : PRIS

Ensuite : **0.79.1. Suivre une autre lettre — de plus en plus vite** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++